

How a neural network learns XOR

A complete beginner guide: read the notebook line by line, follow its calculations, and understand what training actually changes.

The task: receive two bits, each either 0 or 1, and predict whether they are different. A bit is a number with only those two possible values.

First input	Second input	Correct XOR answer	Reason
0	0	0	Both are the same
0	1	1	The inputs differ
1	0	1	The inputs differ
1	1	0	Both are the same



The two class-1 points occupy opposite corners. One straight line cannot separate them from both class-0 points. The hidden layer plus a nonlinear activation allows a more flexible boundary.

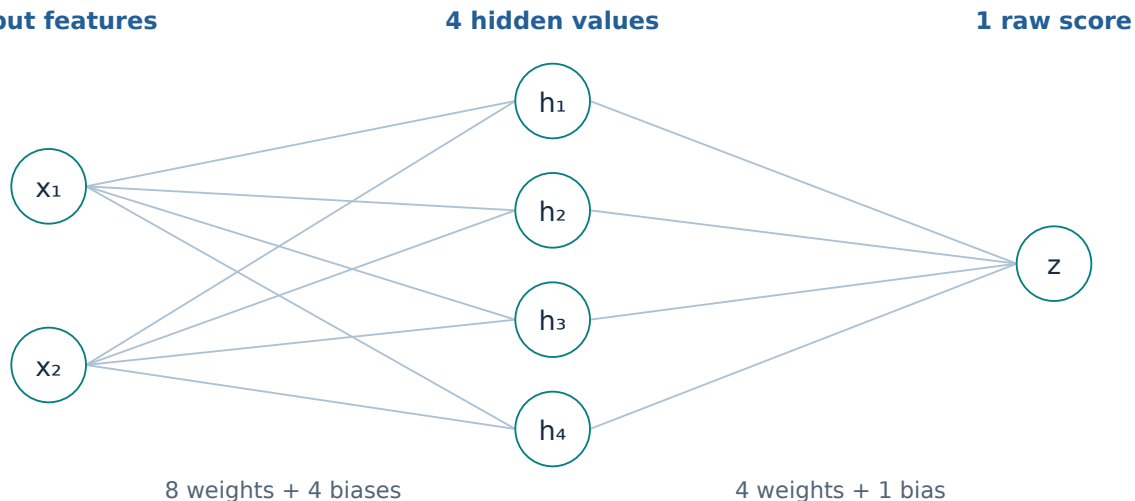
Learning from examples

The network receives the input pairs. The loss calculation receives the correct answers. Training adjusts numbers called **weights and biases**, so the outputs agree with the labels. This is **supervised learning**.

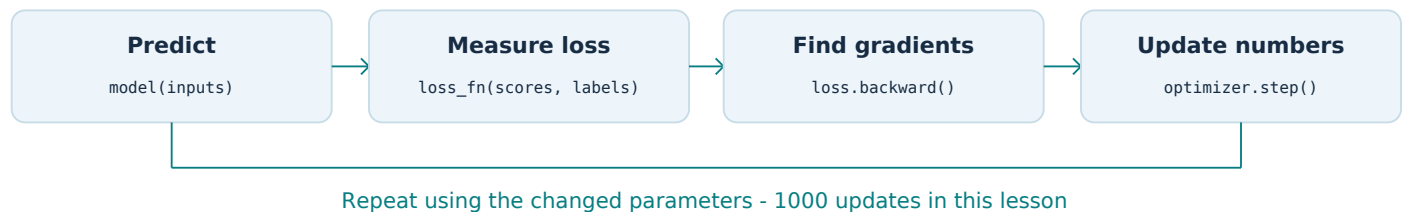
Reading route: pages 2-7 explain the exact notebook. Pages 8-12 show the mathematics with numbers. Pages 13-14 show the verified results and what was learned. Page 15 defines the terminology and suggests experiments.

What is inside the model?

Each connection carries a learned weight. Each calculating neuron also has a learned bias. The diagram shows one input example.



Weight: controls how much an input contributes. **Bias:** an adjustable offset added to a neuron's weighted sum.



Stage	Batch shape	What happens
Input table	(4, 2)	Four rows, two features per row
Linear(2, 4)	(4, 4)	Four weighted sums for each row
Tanh	(4, 4)	Transform each sum nonlinearly
Linear(4, 1)	(4, 1)	One logit per row
Sigmoid at evaluation	(4, 1)	One estimated probability per row

17 learned parameters

First layer: 2×4 weights + 4 biases = 12. Output layer: 4×1 weights + 1 bias = 5. Total: **17 adjustable numbers**. Tanh and sigmoid have no learned parameters in this model.

Code: imports and training data

Load PyTorch, choose repeatable starting values, and create the four input examples with their correct answers.

```
import torch
```

Loads PyTorch, the library that provides tensors, neural-network tools, and automatic gradient calculation.

```
from torch import nn
```

Imports the neural-network toolkit as **nn**. This lets us write nn.Linear rather than torch.nn.Linear.

```
torch.manual_seed(7)
```

Sets PyTorch's pseudo-random seed before the model is built. The SAME seed helps repeat the same starting numbers in the same environment. 7 is arbitrary. Different seeds give different sequences; exact results can vary across versions or devices.

```
inputs = torch.tensor([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
```

Creates a tensor, a table of numbers. Each inner list is one example. Its shape is (4, 2). The notation 0. means 0.0, a floating-point number.

```
answers = torch.tensor([[0.], [1.], [1.], [0.]])
```

Creates the four labels in matching row order. The nested lists make the shape (4, 1), matching the model's output. Labels are floating-point values because this loss expects them.

```
print('Είσοδοι:', inputs.tolist())
```

Prints “Inputs” followed by the values. tolist() converts a tensor into ordinary Python lists for display.

```
print('Σωστές απαντήσεις:', answers.flatten().tolist())
```

Prints “Correct answers.” flatten() turns shape (4, 1) into (4,) for display; it does not change answers itself. tolist() then creates a Python list.

Python background: = assigns a value to a name. Square brackets form lists. Parentheses call a function. A dot accesses a tool or method, such as torch.tensor or inputs.tolist. Run notebook cells in order from top to bottom.

Code: layers, loss, and optimizer

Building a model chooses its structure and initial settings; it does not train it yet.

```
model = nn.Sequential(
```

Starts an ordered sequence of layers. `nn.Sequential` passes each stage's output into the next stage.

```
    nn.Linear(2, 4),
```

Creates four hidden neurons. Each reads both input features. It initializes eight weights and four biases. For one row, two input values become four weighted sums.

```
    nn.Tanh(),
```

Applies Tanh to each hidden value. It maps real numbers into the interval between -1 and 1. Its nonlinear shape allows this network to represent XOR.

```
    nn.Linear(4, 1)
```

Creates one output neuron that combines the four hidden values. It adds four weights and one bias. Its result is a raw score, called a logit, not a probability.

```
)
```

Closes the Sequential constructor. The complete model is now stored under the name `model`.

```
loss_fn = nn.BCEWithLogitsLoss()
```

Creates binary cross-entropy loss for raw logits. It combines sigmoid and binary cross-entropy internally for numerical stability. By default it averages the four example losses. Do not add sigmoid before passing model outputs to this loss.

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.1)
```

Creates Adam, the algorithm that updates the model's weights and biases. `model.parameters()` supplies those numbers. `lr=0.1` is the learning rate, which scales updates. It is not a probability and not a guaranteed improvement per step.

Why an activation is necessary

Two linear layers with no nonlinear activation still act like a single linear transformation. Adding more linear calculations alone cannot solve the XOR separation problem. Tanh supplies the needed nonlinearity.

Code: the training loop

A step uses all four examples, measures loss, computes gradients, then updates parameters.

```
loss_history = []
```

Creates an empty Python list that will store the loss measured at each step. The history is optional; it is useful for drawing a learning curve.

```
for step in range(1000):
```

Repeats the indented instructions 1000 times. `range(1000)` gives step values 0 through 999. Every step uses the full dataset, so each is also one epoch in this example.

```
optimizer.zero_grad()
```

Clears old gradients. PyTorch normally accumulates gradients, so we reset them before calculating those for the current step.

```
scores = model(inputs)
```

Forward pass: all four input rows move through Linear → Tanh → Linear. The scores tensor has shape (4, 1). The model only receives inputs, not the answers.

```
loss = loss_fn(scores, answers)
```

Compares the scores with the labels and returns one mean loss value. PyTorch records the calculation so it can differentiate it later.

```
loss_history.append(loss.item())
```

`item()` extracts the loss as a plain Python number. `append()` adds it to the history. This is the loss BEFORE the parameter update below.

```
loss.backward()
```

Backward pass: calculates gradients of the loss with respect to every learned parameter. A gradient measures sensitivity to a small change. `backward()` does not change the weights itself.

```
optimizer.step()
```

Adam uses the newly computed gradients and its stored history to update all parameters. An individual update is not guaranteed to lower loss.

```
if (step + 1) % 200 == 0:
```

`step + 1` turns zero-based counting into 1-1000. `%` gives the division remainder. `==` compares values. This condition is true every 200 steps.

```
print(f'Βήμα {step + 1}: σφάλμα = {loss.item():.4f}')
```

Prints progress: step 200, 400, 600, 800, and 1000. The f-string inserts values from braces. `:.4f` rounds the loss to four digits after the decimal point.

Code: probabilities and accuracy

Training is finished; we now calculate predictions without updating weights.

```
model.eval()
```

Selects evaluation mode. Some layers, such as dropout, behave differently in this mode. Linear and Tanh do not, so it has no numerical effect in this particular model. It does not disable gradients by itself.

```
with torch.no_grad():
```

Temporarily disables gradient tracking inside the indented with block. We only need predictions, so this avoids unnecessary calculation and memory use.

```
probabilities = torch.sigmoid(model(inputs))
```

`model(inputs)` produces fresh logits after the final update. Sigmoid transforms each logit into an estimated probability of class 1. The shape remains (4, 1).

```
predictions = (probabilities >= 0.5).int()
```

Compares each probability with 0.5. Results are True or False. `int()` converts them to 1 or 0. A probability exactly equal to 0.5 is classified as 1 by this rule.

```
correct = (predictions == answers.int()).sum().item()
```

Converts labels to integers, compares predicted and expected classes, then sums the True matches. `item()` extracts that count as a Python number. The verified count is 4.

```
accuracy = correct / len(answers)
```

`len(answers)` is 4, the number of examples. `correct / 4` gives 1.0. Formatting it as a percentage later displays 100%.

$$\text{accuracy} = \frac{\text{correct classifications}}{\text{number of examples}} = \frac{4}{4} = 1 = 100\%$$

Accuracy and probability answer different questions

A probability is an estimate for one example. Accuracy is the fraction of examples classified correctly after applying a threshold. A model can have 100% accuracy on these four rows while its probabilities are still far from exact 0 and 1.

Important: these are the same rows used during training. For XOR bits they cover the entire input domain, but this is not an independent test of broader generalization.

Code: reading the printed results

Read each prediction and the overall accuracy. Long code lines wrap on the page for readability.

```
for pair, answer, probability, prediction in zip(inputs, answers, probabilities, predictions):
```

zip groups matching rows from inputs, answers, probabilities, and predictions. The for loop processes those groups one at a time. Each name receives one tensor row.

```
    print(f'{pair.tolist()} -> σωστό: {int(answer.item())}, πρόβλεψη: {int(prediction.item())},  
    πιθανότητα 1: {probability.item():.3f}')
```

Prints the input, correct label, predicted class, and estimated probability of 1. tolist() makes the input readable. item() extracts one number. int() shows labels as whole numbers. :.3f displays probabilities with three decimal places.

```
print(f'Ακρίβεια στα 4 παραδείγματα: {correct}/4 ({accuracy:.0%})')
```

Prints the overall accuracy. {correct}/4 shows the number correct out of four. :.0% converts accuracy to a percentage with no decimal places. The literal /4 is correct for this fixed four-row dataset.

Output text	English meaning
Είσοδοι	Inputs
Σωστές απαντήσεις / σωστό	Correct answers / expected label
Βήμα / σφάλμα	Training step / loss
πρόβλεψη	Predicted class
πιθανότητα 1	Estimated probability of class 1
Ακρίβεια στα 4 παραδείγματα	Accuracy on the four examples

Rounding hides small differences

Printed 1.000 does not imply an exact probability of 1. Printed 0.000 does not imply an exact probability of 0. For example, 0.999674 becomes 1.000 when rounded to three decimals. For more detail, change :.3f to :.6f.

A forward pass, with real numbers

Worked example: input [0, 1], after the verified run's 1000 updates. Parameters below are rounded to six decimals for reading.

1. Calculate each hidden neuron

$$a_j = w_{j1} \cdot 0 + w_{j2} \cdot 1 + b_j = w_{j2} + b_j, \quad h_j = \tanh(a_j)$$

Unit	w for x ₁	w for x ₂	Bias b	Sum a	Tanh h
1	-3.782842	-4.029801	0.732643	-3.297158	-0.997267
2	5.011855	-3.484229	0.997869	-2.486360	-0.986247
3	-3.446357	4.625228	1.543578	6.168806	0.999991
4	-3.413564	5.439404	1.152243	6.591647	0.999996

Example, neuron 1: $(-3.782842 \times 0) + (-4.029801 \times 1) + 0.732643 = \mathbf{-3.297158}$. Tanh turns this into approximately **-0.997267**. A negative hidden value is allowed; it is not a probability.

2. Combine those four hidden values

Hidden neuron	Output weight v	Hidden value h	Contribution v × h
1	-4.663178	-0.997267	4.650436
2	-8.204080	-0.986247	8.091246
3	-4.149852	0.999991	-4.149815
4	-4.909929	0.999996	-4.909911

$$z = \sum_{j=1}^4 v_j h_j + c$$

Σ means add the four contributions: their sum is 3.681956. Add output bias 4.347583. The resulting raw score is approximately **z = 8.029539**. Small last-digit differences come from rounding and floating-point arithmetic.

3. Convert score to probability, then decide

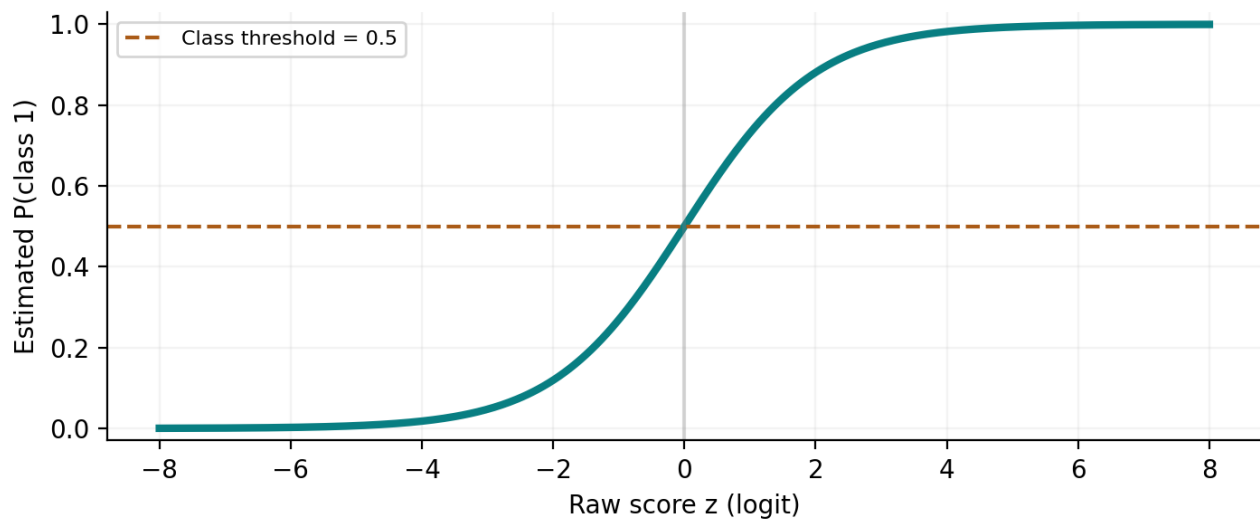
Sigmoid(8.029539) ≈ **0.999674**, or **99.9674%** for class 1. This is at least 0.5, so the predicted XOR answer is **1**. The correct label for [0,1] is also 1.

Where do the percentages come from?

The output neuron gives a raw score. Sigmoid maps that score to an estimated probability; multiplying by 100 gives a percentage.

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad \text{percentage} = 100p$$

Here z is the logit, p is the estimated probability of class 1, and $e \approx 2.71828$ is a mathematical constant. The exponential e raised to a power makes this a smooth, curved transformation.



Raw score z	Estimated $P(1)$	Percentage for class 1	Class at threshold 0.5
-2	0.119203	11.92%	0
0	0.500000	50.00%	1
2	0.880797	88.08%	1
8.029539	0.999674	99.97%	1

A numerical example

For $z = 2$: $e^{-2} \approx 0.135335$. So $p = 1 / (1 + 0.135335) \approx 0.880797$. Multiply by 100: **88.08%** for class 1. The estimate for class 0 is $1 - p \approx$ **11.92%**. These are illustrative values, not that row's final output.

Training produces the scores

Sigmoid itself has no trainable numbers. Training changes the weights and biases upstream, which changes z and therefore p . A high estimated probability is not a guarantee of correctness or evidence that the estimates are well calibrated.

How the loss measures mistakes

Binary cross-entropy penalizes disagreement with the known label. The notebook uses the average over all four examples.

$$\ell(y, p) = -[y \ln p + (1 - y) \ln(1 - p)]$$

$$L = \frac{\ell_1 + \ell_2 + \ell_3 + \ell_4}{4}$$

y is the correct answer, either 0 or 1. p is the estimated probability of 1. $\ln$ is the natural logarithm. If $y = 1$, the loss simplifies to $-\ln(p)$. If $y = 0$, it becomes $-\ln(1 - p)$.

Correct label y	Estimated $P(1)$	Per-example loss	Interpretation
1	0.9	0.105361	Confident and correct
1	0.5	0.693147	Undecided
1	0.1	2.302585	Confident and wrong
0	0.1	0.105361	Confident and correct
0	0.8	1.609438	Confident and wrong

The verified run, before its first update

Input	Correct y	Initial $P(1)$	Per-example loss
[0,0]	0	0.522473	0.739134
[0,1]	1	0.525113	0.644142
[1,0]	1	0.527052	0.640456
[1,1]	0	0.533574	0.762656

Average = $(0.739134 + 0.644142 + 0.640456 + 0.762656) / 4 \approx \mathbf{0.696597}$. Loss is neither accuracy nor a percentage of wrong answers.

BCEWithLogitsLoss receives logits, but mathematically measures this same binary cross-entropy. Combining sigmoid and loss internally avoids unstable intermediate calculations. All loss values here are computed, not read off an illustrative animation.

What happens during backward()?

Backpropagation uses calculus to find how a small change to each of the 17 parameters would affect the loss. It computes gradients.

$$\frac{\partial L}{\partial z_i} = \frac{p_i - y_i}{4}$$

The symbol ∂ means a derivative: sensitivity to a small change. The subscript i refers to one of the four examples. Division by 4 comes from averaging their losses. A positive gradient suggests decreasing that parameter locally; a negative one suggests increasing it.

Initial input	p - y	Gradient of mean loss for z
[0,0]	0.522473	0.130618
[0,1]	-0.474887	-0.118722
[1,0]	-0.472948	-0.118237
[1,1]	0.533574	0.133393

One real parameter: the output bias c

$$\frac{\partial L}{\partial c} = \sum_{i=1}^4 \frac{p_i - y_i}{4}$$

Adding the four entries gives **0.027052864**. Initially the model slightly overpredicts class 1 overall, so the gradient for c is positive. An update that decreases c can reduce the loss locally.

The gradient also flows through hidden neurons

$$\frac{\partial L}{\partial v_j} = \sum_i \frac{p_i - y_i}{4} h_{ij}$$

$$\frac{\partial L}{\partial w_{jk}} = \sum_i \frac{p_i - y_i}{4} v_j (1 - h_{ij}^2) x_{ik}$$

v is an output weight, w is an input-to-hidden weight, and h is a hidden value. The factor $(1 - h^2)$ is Tanh's derivative. Multiplying these sensitivities is the **chain rule**. PyTorch calculates it automatically.

Two separate jobs

loss.backward() calculates and stores gradients. **optimizer.step()** uses them to update parameters. Calculating the gradients and applying an update are separate operations.

How Adam changes the weights

Adam remembers the recent direction and size of gradients, then scales each parameter's update. Here is its first update to the output bias.

A learning rate is a scale for updates. In simple gradient descent the update is $\theta \leftarrow \theta - \eta g$, where θ is a parameter, η is the learning rate, and g is its gradient. **The notebook uses Adam, whose update is different.**

$$m_t = 0.9m_{t-1} + 0.1g_t, \quad s_t = 0.999s_{t-1} + 0.001g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - 0.9^t}, \quad \hat{s}_t = \frac{s_t}{1 - 0.999^t}$$

$$\theta_t = \theta_{t-1} - 0.1 \frac{\hat{m}_t}{\sqrt{\hat{s}_t + 10^{-8}}}$$

t is the update number. m averages recent gradients; s averages their squares. Both start at zero. The hats indicate a correction for that starting point. The tiny 10^{-8} term helps avoid division by zero. These are Adam's defaults with this notebook's $\text{lr}=0.1$.

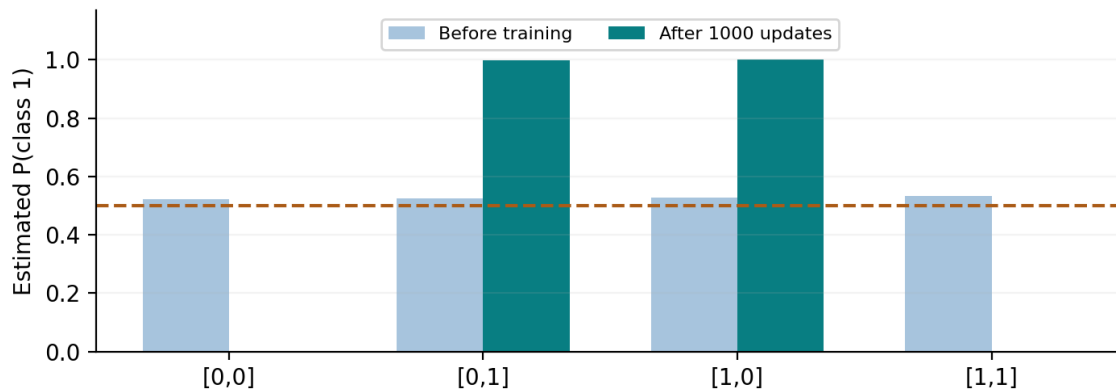
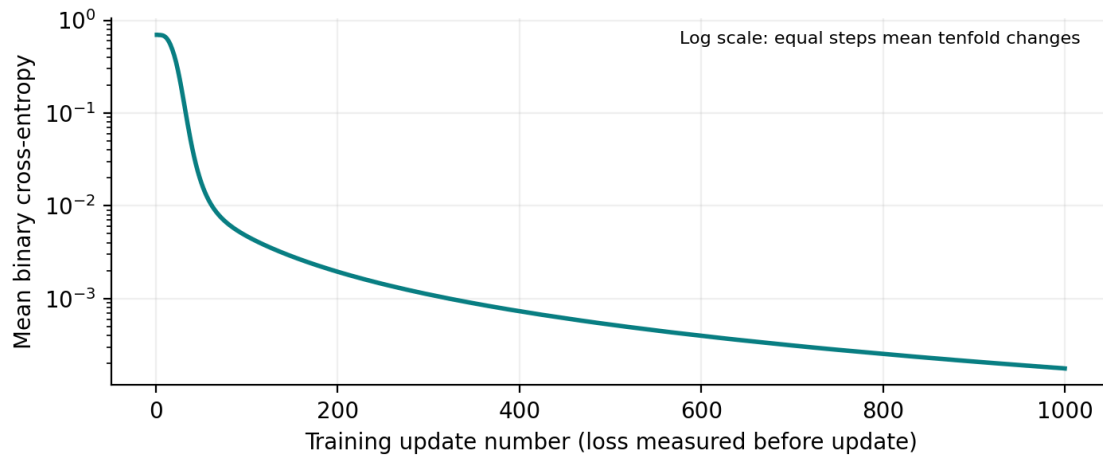
First update, $t = 1$	Numerical value	Meaning
Initial output bias c	0.220380425	Parameter before training
Gradient g	0.027052864	Calculated by backward()
$m_1 = 0.1g$	0.002705286	Average gradient, starting at zero
$s_1 = 0.001g^2$	0.000000732	Average squared gradient
Corrected $\hat{m}_1$ and $\hat{s}_1$	0.027052864; 0.000731857	At $t=1$ they equal g and g^2
Update size	0.099999964	Approximately 0.1 in this first step
New output bias c	0.120380461	Observed after optimizer.step()

A useful comparison

Simple gradient descent would change c by only $0.1 \times 0.027053 \approx 0.002705$. Adam's first update is about 0.1 because it normalizes by the gradient magnitude. Later updates depend on accumulated history. No method guarantees every step improves loss.

What happened in the verified run?

Exact notebook cells were executed in order: seed 7, 2 → 4 → 1 layers, Tanh, Adam at 0.1, and 1000 full-batch updates.



Input	Correct	Final estimated P(1)	Percentage for class 1	Class
[0,0]	0	0.000003330	0.000333%	0
[0,1]	1	0.999674439	99.967444%	1
[1,0]	1	0.999929428	99.992943%	1
[1,1]	0	0.000304660	0.030466%	0

Training accuracy: 4/4 = 100%. **Initial loss:** 0.696597040. **Step-1000 logged loss:** 0.000176213, measured before the last update. **Loss recomputed after it:** 0.000176051. Both final values print as 0.0002 with four decimals.

What did the model actually learn?

It learned 17 weights and biases that implement a nonlinear function matching the complete binary XOR truth table at the chosen threshold.

Hidden neuron	Learned w for x_1	Learned w for x_2	Learned hidden bias
1	-3.782842	-4.029801	0.732643
2	5.011855	-3.484229	0.997869
3	-3.446357	4.625228	1.543578
4	-3.413564	5.439404	1.152243

Output weights for h_1, h_2, h_3, h_4	Output bias c
-4.663178, -8.204080, -4.149852, -4.909929	4.347583

$$p(x) = \sigma(v^T \tanh(Wx + b) + c)$$

W groups the input-to-hidden weights. x is the input pair. b is the vector of hidden biases. v groups the output weights; the superscript T means transpose, used to take a weighted sum. This formula and the learned numbers are the model's stored knowledge.

The result in plain language

For the four valid bit pairs, the model outputs a probability near 1 when inputs differ and near 0 when they are the same. Thresholding at 0.5 reproduces **“different → 1, same → 0.”** That is the XOR mapping.

What this does and does not establish

The four rows exhaust the binary input domain, so matching all four solves binary XOR classification. The model has not demonstrated human understanding, discovered a symbolic program, or learned a general method for all problems. Values such as [0.3,0.7] are outside this lesson's XOR domain. High confidence on the training rows is not evidence of reliability there.

Where does the learning live?

The parameters live in model while the notebook session is running. This notebook does not save them to disk. Rerunning the model-building cell creates new initial parameters. Continuing only the training cell trains the existing model further; restart and run all cells for a fresh experiment.

Hidden neurons learn numerical features that work together. They are not automatically named logical rules such as AND or OR. Different seeds can produce different internal features while giving the same four thresholded answers.

Keep these ideas in mind

A quick vocabulary reference, three experiments, and primary sources for the mathematics and PyTorch behavior.

Term	Beginner meaning
Tensor / shape	A container of numbers / its dimensions, such as four rows by two columns.
Feature / label	Input information / the correct answer used during training.
Parameter / hyperparameter	A learned number, such as a weight / a choice such as learning rate or neuron count.
Activation / logit	A transformation such as Tanh / a raw output score before sigmoid.
Loss / gradient	A measure of mismatch / sensitivity of loss to a small parameter change.
Forward / backward	Calculate outputs / calculate gradients through those calculations.
Optimizer / inference	Rule for updating parameters / using the model to make predictions.
Epoch / batch	One pass through the data / examples processed together. Here all four form one batch.
Training / fine-tuning	Fit new initial parameters / adapt a model that has already been trained.

Try three controlled experiments

1. Remove Tanh, restart, and rerun everything. Explain why the resulting linear function cannot classify all four XOR corners correctly.
2. Reduce training to 100 steps. Compare probabilities, loss, and accuracy; they measure different things.
3. Use 2 or 8 hidden neurons, changing both Linear layers so their sizes agree. Change one choice at a time, then restart for a fair comparison.

Primary sources

Loss definition: [Official PyTorch documentation](#)

Adam update rule: [Official PyTorch documentation](#)

Automatic gradients: [Official PyTorch documentation](#)

Reproducibility: [Official PyTorch documentation](#)